

Problem Set 6 (Graphs - 2)

BM2043 - Algorithms and Data Structure Lab

October 18, 2024

Shortest Path Algorithms and Problems

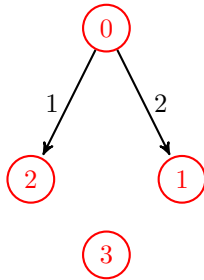
Problem 1: Shortest Path in DAG

Problem Statement:

Given a Directed Acyclic Graph (DAG) of N vertices from 0 to $N - 1$ and M edges, and a 2D integer array $edges$, where there is a directed edge from vertex $edge[i][0]$ to vertex $edge[i][1]$ with a distance of $edge[i][2]$ for all i , find the shortest path from the source vertex (assumed to be 0) to all the vertices. If it is impossible to reach any vertex, then return -1 for that vertex.

Example 1

- **Input:**



$N = 4, M = 2, edges = [[0, 1, 2], [0, 2, 1]]$

- **Output:**

$[0, 2, 1, -1]$

- **Explanation:** Shortest path from 0 to 1 is $0 \rightarrow 1$ with edge weight 2. Shortest path from 0 to 2 is $0 \rightarrow 2$ with edge weight 1. There is no way to reach 3, so it's -1 for 3.

Example 2

- **Input:**

$N = 6, M = 7, edges = [[0, 1, 2], [0, 4, 1], [4, 5, 4], [4, 2, 2], [1, 2, 3], [2, 3, 6], [5, 3, 1]]$

- **Output:**

[0, 2, 3, 6, 1, 5]

- **Explanation:** Shortest path from 0 to 1 is $0 \rightarrow 1$ with edge weight 2. Shortest path from 0 to 2 is $0 \rightarrow 4 \rightarrow 2$ with edge weight 3. Shortest path from 0 to 3 is $0 \rightarrow 4 \rightarrow 5 \rightarrow 3$ with edge weight 6. Shortest path from 0 to 4 is $0 \rightarrow 4$ with edge weight 1. Shortest path from 0 to 5 is $0 \rightarrow 4 \rightarrow 5$ with edge weight 5.

Constraints

- $1 \leq N, M \leq 5 \times 10^4$
- $0 \leq \text{edge}[i][0], \text{edge}[i][1] < N - 1$
- $1 \leq \text{edge}[i][2] < 10^4$

Note A Directed Graph (containing one-sided edges) having no cycles is said to be a Directed Acyclic Graph (DAG).

Class Definition

```
class Solution {
public:
    vector<int> shortestPath(int N, int M, vector<vector<int>>& edges) {
        // Implement the algorithm for finding shortest paths in the DAG here
    }
};
```

Problem 2: Shortest Path in Undirected Graph with Unit Weights

Problem Statement:

Given an undirected graph of N vertices from 0 to $N - 1$ and M edges, represented by a 2D integer array *edges*, where there is an edge from vertex *edge*[*i*][0] to vertex *edge*[*i*][1] of unit weight, find the shortest path from the source (vertex 0) to all other nodes in this graph.

If a vertex is unreachable from the source node, then return -1 for that vertex.

Example 1

- **Input:**

```
n = 9
m = 10
edges = [[0,1],[0,3],[3,4],[4,5],[5,6],[1,2],[2,6],[6,7],[7,8],[6,8]]
```

- **Output:**

```
0 1 2 1 2 3 3 4 4
```

- **Explanation:** The output array shows the shortest path to all nodes from the source vertex (0). $\text{Dist}[0] = 0$, $\text{Dist}[1] = 1$, $\text{Dist}[2] = 2$, ..., $\text{Dist}[8] = 4$. For a node, if the value of $\text{Dist}[\text{node}] = -1$, then that node is unreachable from the source node.

Example 2

- **Input:**

```
n = 8
m = 10
edges = [[1,0],[2,1],[0,3],[3,7],[3,4],[7,4],[7,6],[4,5],[4,6],[6,5]]
```

- **Output:**

```
0 1 2 1 2 3 3 2
```

- **Explanation:** The output array shows the shortest path to all nodes from the source vertex (0).
 $Dist[0] = 0$, $Dist[1] = 1$, $Dist[2] = 2$, ..., $Dist[7] = 2$.

Constraints

- $1 \leq n, m \leq 10^4$
- $0 \leq edges[i][j] \leq n - 1$

Class Definition

```
class Solution {
public:
    vector<int> shortestPath(vector<vector<int>>& edges, int N, int M) {
        // Implement the logic to find the shortest path from source to all nodes
    }
};
```

Dijkstra's Algorithm

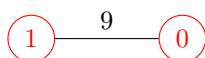
Problem Statement:

Given a weighted, undirected graph with V vertices (numbered from 0 to $V - 1$) and an adjacency list adj , where $adj[i]$ contains a list of lists. Each inner list consists of two integers: the first integer denotes a connected vertex j , and the second integer corresponds to the weight of the edge between i and j .

Given a source node S , find the shortest distance of all vertices from the source vertex S . Return a list of integers denoting the shortest distance between each node and the source vertex S . If a vertex is not reachable from the source, its distance should be represented as 10^9 .

Example 1

- **Input:**



```
V = 2
adj = [[1, 9], [0, 9]]
S = 0
```

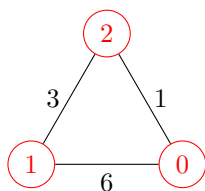
- **Output:**

```
[0, 9]
```

- **Explanation:** The shortest distance from node 0 (source) to node 0 is 0, and the shortest distance from node 0 to node 1 is 9.

Example 2

- **Input:**



$V = 3$

$adj = [[1, 1], [2, 6]], [[2, 3], [0, 1]], [[1, 3], [0, 6]]]$

$S = 2$

- **Output:**

[4, 3, 0]

- **Explanation:** For node 0, the shortest path is $2 \rightarrow 1 \rightarrow 0$ (distance = 4). For node 1, the shortest path is $2 \rightarrow 1$ (distance = 3).

Constraints

- $1 \leq V \leq 10,000$
- $0 \leq adj[i][j] \leq 10,000$
- $1 \leq adj.size() \leq \frac{V \cdot (V-1)}{2}$
- $0 \leq S < V$

Class Definition

```
class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<int>>> adj[], int S) {
        // Implement Dijkstra's algorithm to find shortest paths from source S
    }
};
```

Problem 3: Cheapest Flights Within K Stops

Problem Statement:

You are given n cities connected by flights. Each flight is represented by a 3D array $flights[i] = [from_i, to_i, price_i]$, where there is a flight from city $from_i$ to city to_i with cost $price_i$.

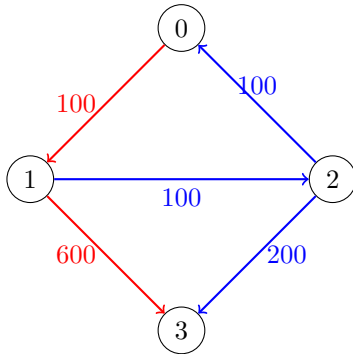
You are also given three integers:

- src (source city),
- dst (destination city), and
- k (the maximum number of stops).

Your task is to return the **cheapest price** from src to dst such that the route has at most k stops. If there is no such route, return -1 .

Example 1:

• **Input:**



```
n = 4
flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]
src = 0, dst = 3, k = 1
```

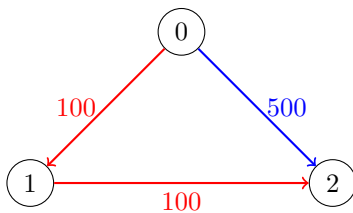
• **Output:**

700

- **Explanation:** The optimal path with at most 1 stop from city 0 to 3 is $0 \rightarrow 1 \rightarrow 3$ with a total cost of $100 + 600 = 700$. Another path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ is cheaper but has 2 stops, which exceeds the limit.

Example 2:

• **Input:**



```
n = 3
flights = [[0,1,100],[1,2,100],[0,2,500]]
src = 0, dst = 2, k = 1
```

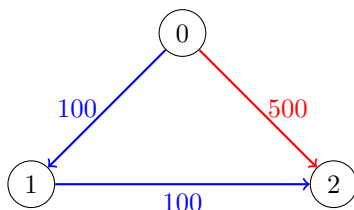
• **Output:**

200

- **Explanation:** The optimal path with at most 1 stop from city 0 to 2 is $0 \rightarrow 1 \rightarrow 2$ with a total cost of $100 + 100 = 200$.

Example 3:

- **Input:**



```
n = 3
flights = [[0,1,100],[1,2,100],[0,2,500]]
src = 0, dst = 2, k = 0
```

- **Output:**

500

- **Explanation:** The optimal path with no stops from city 0 to 2 is a direct flight costing 500.

Constraints:

- $1 \leq n \leq 100$
- $0 \leq \text{flights.length} \leq \frac{n \cdot (n-1)}{2}$
- Each flight is represented by an array of length 3, where:
 - $0 \leq \text{from}_i, \text{to}_i < n$
 - $\text{from}_i \neq \text{to}_i$
 - $1 \leq \text{price}_i \leq 10^4$
- No multiple flights exist between the same pair of cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

Class Definition:

```
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        // Implement Bellman-Ford or Dijkstra's algorithm to find the shortest path
    }
};
```

Problem 4: Path with Minimum Effort

Problem Statement:

You are a hiker preparing for an upcoming hike. You are given *heights*, a 2D array of size *rows* × *columns*, where *heights*[*row*][*col*] represents the height of cell (*row*, *col*). You start at the top-left cell (0,0) and need to travel to the bottom-right cell (*rows* − 1, *columns* − 1).

You can move **up**, **down**, **left**, or **right**, and you wish to find a route that requires the **minimum effort**.

- A route's effort is defined as the **maximum absolute difference in heights** between two consecutive cells of the route. - Return the **minimum effort** required to travel from the top-left to the bottom-right.

Example 1:

- **Input:**

1	2	2
3	8	2
5	3	5

`heights = [[1,2,2],[3,8,2],[5,3,5]]`

- **Output:**

2

- **Explanation:** The route $[1, 3, 5, 3, 5]$ has a maximum absolute difference of 2 between consecutive cells. This is better than the route $[1, 2, 2, 2, 5]$, where the maximum absolute difference is 3.

Example 2:

- **Input:**

1	2	3
3	8	4
5	3	5

`heights = [[1, 2, 3], [3, 8, 4], [5, 3, 5]]`

- **Output:**

1

- **Explanation:** The route $[1, 2, 3, 4, 5]$ has a maximum absolute difference of 1 between consecutive cells, which is better than the route $[1, 3, 5, 3, 5]$.

Example 3:

- **Input:**

1	2	1	1	1
1	2	1	2	1
1	2	1	2	1
1	2	1	2	1
1	2	1	2	1
1	1	1	2	1

```
heights = [[1, 2, 1, 1, 1], [1, 2, 1, 2, 1], [1, 2, 1, 2, 1],
            [1, 2, 1, 2, 1], [1, 1, 1, 2, 1]]
```

• **Output:**

0

- **Explanation:** This route does not require any effort, as there is no change in height.

Constraints:

- $rows == heights.length$
- $columns == heights[i].length$
- $1 \leq rows, columns \leq 100$
- $1 \leq heights[i][j] \leq 10^6$

Class Definition:

```
class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        // Implement your solution here (Dijkstra-like approach or Binary Search with BFS/DFS)
    }
};
```

Bellman-Ford Algorithm

Problem Statement:

Given a weighted and directed graph with V vertices and E edges. Each edge is represented as $[a_i, b_i, w_i]$, meaning there is a directed edge from vertex a_i to vertex b_i with weight w_i .

Your task is to find the shortest distance from the source vertex S to all other vertices. If a vertex cannot be reached from S , its distance should be marked as 10^9 .

If the graph contains a negative cycle, return -1 .

Examples: Example 1:

- **Input:**

$V = 6$, Edges = $[[3, 2, 6], [5, 3, 1], [0, 1, 5], [1, 5, -3],$
 $[1, 2, -2], [3, 4, -2], [2, 4, 3]]$, $S = 0$

- **Output:**

0 5 3 3 1 2

- **Explanation:** Shortest paths from node 0:

- $0 \rightarrow 1$ (distance = 5)
- $0 \rightarrow 1 \rightarrow 2$ (distance = 3)
- $0 \rightarrow 1 \rightarrow 5 \rightarrow 3$ (distance = 3)
- $0 \rightarrow 1 \rightarrow 5 \rightarrow 3 \rightarrow 4$ (distance = 1)
- $0 \rightarrow 1 \rightarrow 5$ (distance = 2)

Example 2:

- **Input:**

$V = 2$, Edges = $[[0, 1, 9]]$, $S = 0$

- **Output:**

0 9

- **Explanation:** Shortest path from node 0 to node 1 is $0 \rightarrow 1$ (distance = 9).

Constraints:

- $1 \leq V \leq 500$
- $1 \leq E \leq V \times (V - 1)$
- $-1000 \leq \text{edges}[i][2] \leq 1000$ (Note: Fixed index to 2 for the weight of the edges)
- $0 \leq S < V$

Class Definition:

```
class Solution {
public:
    vector<int> bellman_ford(int V, vector<vector<int>>& edges, int S) {
        // Code here
    }
};
```

Floyd-Warshall Algorithm

Problem Statement:

Given a graph of V vertices numbered from 0 to $V - 1$, find the shortest distances between every pair of vertices in a given edge-weighted directed graph. The graph is represented as an adjacency matrix of size $n \times n$.

- **Matrix[i][j]** denotes the weight of the edge from i to j . If **Matrix[i][j] = -1**, it means there is no edge from i to j .

Examples: Example 1:

- **Input:**

```
matrix = [[0, 2, -1, -1],
          [1, 0, 3, -1],
          [-1, -1, 0, 1],
          [3, 5, 4, 0]]
```

- **Output:**

```
[[0, 2, 5, 6],
 [1, 0, 3, 4],
 [4, 6, 0, 1],
 [3, 5, 4, 0]]
```

- **Explanation:** The distance from vertex 0 to vertex 0 is 0, the distance from vertex 0 to vertex 1 is 2, and so on.

Example 2:

- **Input:**

```
matrix = [[0, 25],
          [-1, 0]]
```

- **Output:**

```
[[0, 25],
 [-1, 0]]
```

- **Explanation:** The matrix already contains the shortest distance.

Constraints:

- $1 \leq n \leq 100$
- $-1 \leq matrix[i][j] \leq 1000$

Class Definition:

```
class Solution {
public:
    int findCity(int n, int m, vector<vector<int>>& edges,
                int distanceThreshold) {
        // code here
    }
};
```

Problem 5: Find the City With the Smallest Number of Neighbors at a Threshold Distance

Problem Statement:

You are given n cities numbered from 0 to $n-1$. An array `edges` is provided where `edges[i] = [from_i, to_i, weight_i]` represents a bidirectional and weighted edge between cities `from_i` and `to_i`. You are also given an integer `distanceThreshold`.

Your task is to return the city with the **smallest number of cities** that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the **greatest number**.

The distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Examples: Example 1:

- **Input:**

```
n = 4
edges = [[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]
distanceThreshold = 4
```

- **Output:**

3

- **Explanation:**

- City 0 -> [City 1, City 2]
- City 1 -> [City 0, City 2, City 3]
- City 2 -> [City 0, City 1, City 3]
- City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at `distanceThreshold = 4`, but city 3 has the greatest number.

Example 2:

- **Input:**

```
n = 5
edges = [[0, 1, 2], [0, 4, 8], [1, 2, 3], [1, 4, 2],
         [2, 3, 1], [3, 4, 1]]
distanceThreshold = 2
```

- **Output:**

0

- **Explanation:**

- City 0 -> [City 1]
- City 1 -> [City 0, City 4]
- City 2 -> [City 3, City 4]
- City 3 -> [City 2, City 4]
- City 4 -> [City 1, City 2, City 3]

City 0 has 1 neighboring city at `distanceThreshold = 2`.

Constraints:

- $2 \leq n \leq 100$
- $1 \leq \text{edges.length} \leq n \times (n - 1)/2$
- `edges[i].length == 3`
- $0 \leq \text{from}_i < \text{to}_i < n$
- $1 \leq \text{weight}_i, \text{distanceThreshold} \leq 10^4$
- All pairs $(\text{from}_i, \text{to}_i)$ are distinct.

Class Definition:

```
class Solution {
public:
    int findTheCity(int n, vector<vector<int>>& edges,
                    int distanceThreshold) {
        // Implement your solution here (using Dijkstra's algorithm
        // or Floyd-Warshall)
    }
};
```

Problem 6: Number of Ways to Arrive at Destination

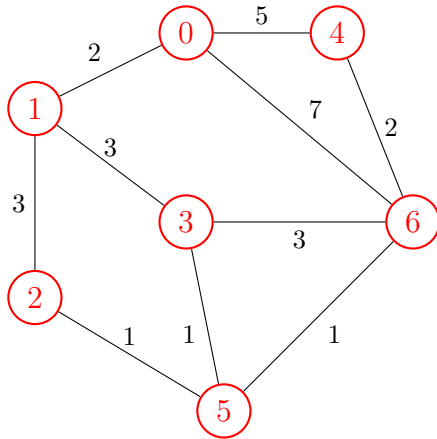
Problem Statement:

A city consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. Given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections u_i and v_i that takes time_i minutes to travel. Your task is to determine the **number of ways** to travel from intersection 0 to intersection $n - 1$ in the **shortest amount of time**.

Since the answer may be large, return it modulo $10^9 + 7$.

Examples: Example 1:

• Input:



$n = 7,$

roads = $[[0,6,7], [0,1,2], [1,2,3], [1,3,3], [6,3,3], [3,5,1], [6,5,1], [2,5,1], [0,4,5], [4,6,2]]$

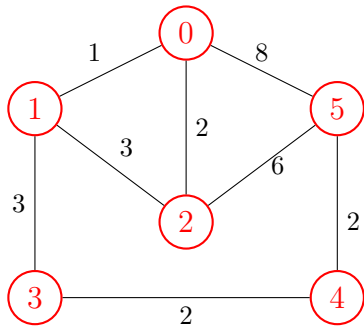
• Output: 4

• Explanation: The four ways to get there in 7 minutes (the shortest calculated time) are:

- $0 \rightarrow 6$
- $0 \rightarrow 4 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Example 2:

• Input:



$n = 6,$

roads = $[[0,5,8], [0,2,2], [0,1,1], [1,3,3], [1,2,3], [2,5,6], [3,4,2], [4,5,2]]$

• Output: 3

• Explanation: The three ways to get there in 8 minutes (the shortest calculated time) are:

- $0 \rightarrow 5$
- $0 \rightarrow 2 \rightarrow 5$
- $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Constraints:

- $1 \leq n \leq 200$
- $n - 1 \leq \text{roads.length} \leq \frac{n \times (n-1)}{2}$
- $\text{roads}[i].\text{length} == 3$
- $0 \leq u_i, v_i \leq n - 1$
- $1 \leq \text{time}_i \leq 10^9$
- $u_i \neq v_i$

Class Definition:

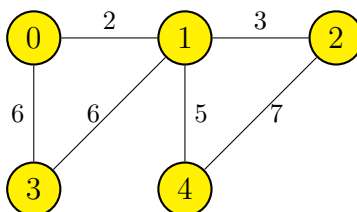
```
class Solution {
public:
    int countPaths(int n, vector<vector<int>> &roads) {
        // Implement your solution here
    }
};
```

Spanning Tree

A spanning tree is a subset of a weighted graph in which there are N nodes (i.e., all the nodes present in the original graph) and $N - 1$ edges, and all nodes are reachable from each other.

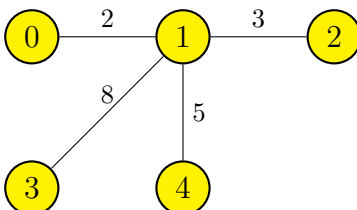
Examples

Assume we are given an undirected weighted graph with N nodes and M edges. Here, in the example, $N = 6$ and $M = 7$ as shown in the figure:



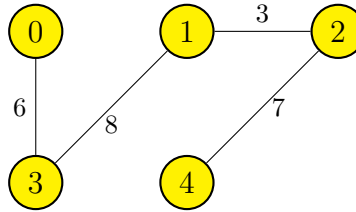
Given Undirected Weighted Graph

For the above graph, one of the spanning trees can be:

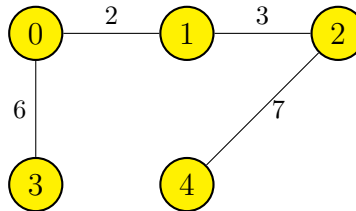


Spanning Tree I and Sum of edge weights = 18

Two more spanning trees for the above graph are:



Spanning Tree II and Sum of edge weights = 24



Spanning Tree III and Sum of edge weights = 18

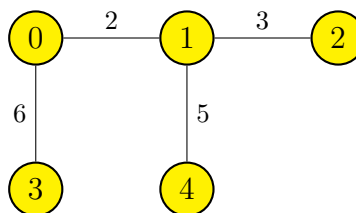
All the above spanning trees are subgraphs of the main graph, containing all the N nodes and $N - 1$ edges to connect all nodes. For a particular spanning tree, the sum of edge weights of all its edges is called the *Weight of that spanning tree*.

Note: A graph may have multiple spanning trees as shown in the above example.

Minimum Spanning Tree

Among all possible spanning trees of a graph, the minimum spanning tree is the one for which the sum of all the edge weights is the minimum.

The three spanning trees shown for the above graph have weights of 18, 24, and 18 respectively. If all possible spanning trees are drawn, it will be found that the following spanning tree with the minimum sum of edge weights, 16, is the minimum spanning tree for the above graph:



Minimum Spanning Tree and Sum of edge weights = 16

Note: A graph can have multiple minimum spanning trees.

Algorithms for finding the Minimum Spanning Tree

There are two algorithms that come into play when finding the minimum spanning tree for a given graph:

- **Prim's Algorithm:** A greedy algorithm that grows the minimum spanning tree one edge at a time.
- **Kruskal's Algorithm:** Another greedy algorithm that adds edges in increasing order of their weight.

Use Cases

The concept of finding the minimum spanning tree for a given graph plays a huge role in daily life:

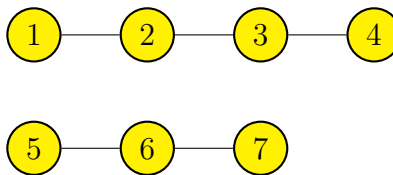
- **Telecommunications Networks:** Determining the most efficient way to lay out cables to connect all phone exchanges with the minimum total cable length.
- **Computer Networks:** Finding the optimal way to connect multiple computers or routers in a Local Area Network (LAN) or Wide Area Network (WAN) to minimize the amount of networking hardware and cabling needed.
- **Transportation Networks:** Creating the most cost-effective road/railway network that connects all cities without redundant routes.
- **Water Supply Networks:** Ensuring an efficient layout of pipelines that connect all areas with the least possible construction cost and maintenance effort.

Disjoint Set

The Disjoint Set data structure is a crucial topic in graph theory. To understand its necessity, consider the following problem:

Problem Statement

Given two components of an undirected graph, determine if node 1 and node 5 belong to the same component.



Graph 1: Components of the Graph

Approach

To solve this problem, Depth-First Search (DFS) or Breadth-First Search (BFS) traversal techniques can be employed. By traversing the graph components, it is evident that node 1 and node 5 are in different components. This brute-force approach has a time complexity of $O(N + E)$ where N is the number of nodes and E is the number of edges. However, using a Disjoint Set data structure, this problem can be solved in constant time.

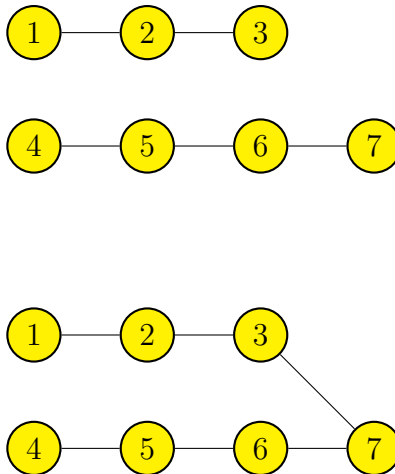
The Disjoint Set data structure is particularly useful for dynamic graphs.

Dynamic Graph

A dynamic graph refers to a graph that continuously changes its configuration. For example, consider the edge information for a graph as:

$$\{\{1, 2\}, \{2, 3\}, \{4, 5\}, \{6, 7\}, \{5, 6\}, \{3, 7\}\}$$

Adding edges one by one changes the structure of the graph at each step. After adding the first four edges, nodes 4 and 1 belong to different components. After adding all six edges, nodes 4 and 1 belong to the same component.



Graph 2: Adding edges dynamically

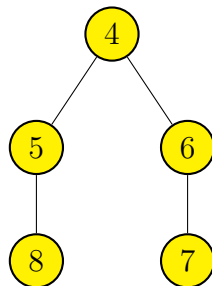
Functionalities of Disjoint Set Data Structure

The Disjoint Set data structure provides two primary functionalities:

- **Finding the parent (ultimate parent) of a particular node.**
- **Union operation (which adds an edge between two nodes):**
 - Union by Rank
 - Union by Size

Terminologies

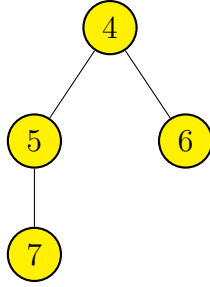
- **Ultimate Parent:** The parent of a node refers to the node directly above that particular node. The ultimate parent refers to the topmost node or the root node of that component.



Graph 3: Graph showing ultimate parent

Union by Rank

Rank: The rank of a node refers to the distance (the number of nodes including the leaf node) between the furthest leaf node and the current node. Rank includes all nodes beneath the current node.



Graph 4: Example of union by rank

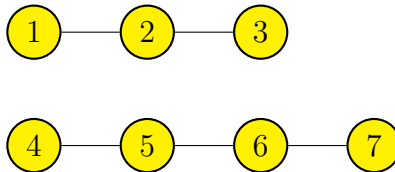
Implementation

To implement Union by Rank, two arrays of size N (number of nodes) are needed:

- **Rank array:** Stores the rank of each node.
- **Parent array:** Stores the ultimate parents of each node.

Algorithm

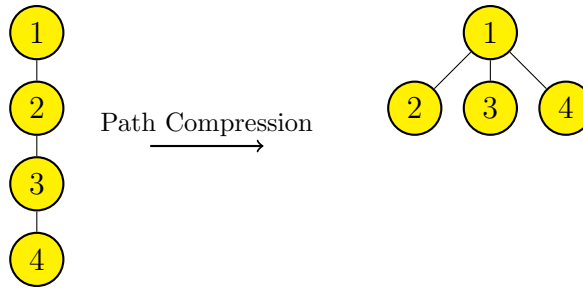
- **Initial Configuration:**
 - Rank array: Initialized with zeros.
 - Parent array: Initialized with the value of nodes, i.e., $parent[i] = i$.
- **Union Function Steps:**
 - The Union function takes two nodes (let's say u and v) as arguments.
 - Find the ultimate parent of u and v using a helper `findPar()` function. Let pu and p_v be the ultimate parents of u and v , respectively.
 - Determine the rank of pu and p_v .
 - Connect the ultimate parent with a smaller rank to the other ultimate parent with a larger rank. If the ranks are equal, connect any parent to the other and increase the rank of the parent node to which the other is connected.



Graph 5: Union by Rank with edges

Path Compression

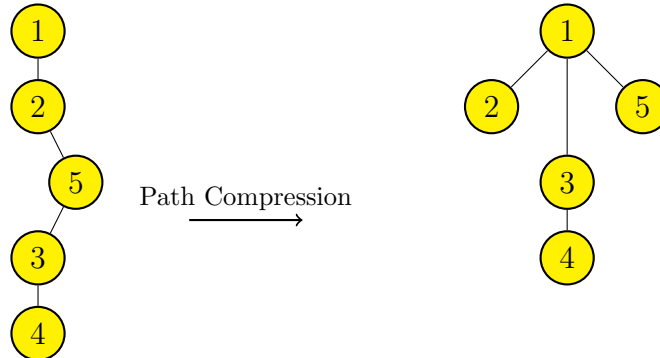
Path Compression connects each node in a path to its ultimate parent, reducing the time complexity nearly to constant time. The path from the node to its ultimate parent is compressed to only a single edge.



Graph 6: Path compression example

How Time Complexity Reduces

Before path compression, finding the ultimate parent for node 4 involves traversing back to node 1, which is a height of size $\log N$. After path compression, the ultimate parent is accessed in a single step, reducing traversal and thus time complexity.



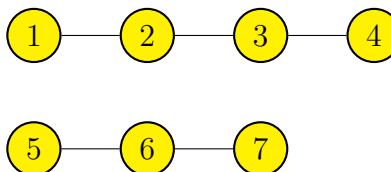
Graph 7: Path compression with rank unchanged

Note: We cannot change the ranks while applying path compression.

Overall, the `findPar()` method helps reduce the time complexity of the union-by-rank method as it can find the ultimate parent within constant time.

Union by Size

Union by Size is an alternative to Union by Rank where we track the size of components rather than their depth.



Graph 1: Components of the Graph

Implementation

To implement Union by Size, two arrays of size N (number of nodes) are needed:

- **Size array:** Stores the size of each node, i.e., the size of the component starting from that node.
- **Parent array:** Stores the ultimate parents of each node.

Algorithm

- **Initial Configuration:**

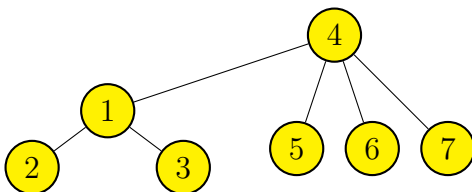
- **Size array:** Initialized with all values as 1.
- **Parent array:** Initialized with the value of nodes, i.e., $parent[i] = i$.

- **Union Function Steps:**

- The Union function takes two nodes (let's say u and v) as arguments.
- Find the ultimate parent of u and v using a helper `findPar()` function. Let pu and p_v be the ultimate parents of u and v , respectively.
- Determine the size of pu and p_v .
- Connect the ultimate parent with the smaller size to the other ultimate parent with a larger size. If the sizes are equal, connect any parent to the other. While connecting in both cases, the size of the parent node (to whom the node was connected) will be increased by the size of the other parent node which is connected.

Example

Given the edges of a graph as $\{\{1, 2\}, \{2, 3\}, \{4, 5\}, \{6, 7\}, \{5, 6\}, \{3, 7\}\}$:



Graph 2: Final configuration of the graph

Note: Union by Size is often seen as more intuitive than Union by Rank because the size metric remains consistent even after applying path compression.

Observations

- In Union by Rank, the actual ranks get distorted during path compression. However, with Union by Size, the sizes remain consistent during the process of path compression.

Conclusion: Union by Size helps maintain the structure of the components more intuitively compared to Union by Rank, especially when path compression is involved.

MST Algorithms

Find the MST Weight

Problem Statement:

You are given a weighted, undirected, and connected graph with V vertices (numbered from 0 to $V - 1$) and E edges. Each edge is represented as a triplet $[a_i, b_i, w_i]$ where a_i and b_i are the vertices connected by the edge, and w_i is the weight of that edge.

Your task is to compute the **sum of the weights of the edges** in the **Minimum Spanning Tree (MST)** of the graph.

A **Minimum Spanning Tree (MST)** is a subset of the edges that connects all the vertices together, without any cycles, and with the minimum possible total edge weight.

Examples: Example 1:

- **Input:**

```
V = 4
edges = [[0, 1, 1], [1, 2, 2], [2, 3, 3], [0, 3, 4]]
```

- **Output:** 6

- **Explanation:** Edges included in the MST:

- [0, 1] with weight 1
- [1, 2] with weight 2
- [2, 3] with weight 3

The total weight of the MST is $1 + 2 + 3 = 6$.

Example 2:

- **Input:**

```
V = 3
edges = [[0, 1, 5], [1, 2, 10], [2, 0, 15]]
```

- **Output:** 15

- **Explanation:** Edges included in the MST:

- [0, 1] with weight 5
- [1, 2] with weight 10

The total weight of the MST is $5 + 10 = 15$.

Constraints:

- $2 \leq V \leq 10^3$
- $V - 1 \leq E \leq 10^4$
- $1 \leq w_i \leq 10^5$

Class Definition:

```
class Solution {
public:
    int spanningTree(int V, vector<vector<int>> adj[]) {
        // Implement your MST solution here (using Kruskal's or Prim's algorithm)
    }
};
```

Key Algorithms to Solve the Problem: To find the **Minimum Spanning Tree (MST)**, you can use either of these algorithms:

- **Kruskal's Algorithm** (using Union-Find/Disjoint Set):
 - Sort the edges by weight.
 - Start adding the smallest edge to the MST if it doesn't form a cycle.
 - Use Union-Find to detect cycles and keep track of connected components.
 - Time Complexity: $O(E \log E)$ due to sorting of edges.
- **Prim's Algorithm** (using a priority queue/heap):
 - Start from any node (usually node 0).
 - Use a priority queue to always expand the vertex that is connected with the smallest edge.
 - Continue until all vertices are included.
 - Time Complexity: $O(E \log V)$ with a priority queue.

Hints: For Kruskal's Algorithm:

- Use a **Union-Find/Disjoint Set** to keep track of which nodes are in the same connected component. Only add edges that connect two different components.
- Sort edges by weight and then add the smallest available edge that doesn't form a cycle.

For Prim's Algorithm:

- Use a **priority queue** to select the edge with the smallest weight connecting a visited node to an unvisited node.
- Keep track of which nodes have been visited and continue expanding the MST until all vertices are covered.